Ders 11 Genişletilmiş Notu: Kalite Tekrarı, Yazılım Proje Yönetimi, Proje Planı ve Risk Yönetimi

Ders 11 Genişletilmiş Notu: Kalite Tekrarı, Yazılım Proje Yönetimi, Proje Planı ve Risk Yönetimi

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Yazılım Mühendisliğinin Tanımına Dönüş
- 2. Yazılım Kalite Faktörleri ve McCall Modeli
- 3. Kaliteyi Sağlamanın Yolları
- 4. Test Stratejisi ve Test Tipleri
- 5. Yazılım Mühendisliğinin Bölümleri ve Proje Yönetiminin Önemi
- 6. Proje Yönetiminin Aşamaları
- 7. Planlama İlkeleri ve Ölçüm
- 8. Yazılım Proje Planının İçeriği
- 9. Risk Yönetimi
- 10. Yazılım Ölçütleri (Software Metrics)
- 11. İş Planı ve İş Kırılım Yapısı
- 12. Ayrıntılı Proje Planı Şablonu
- 13. Gantt Diyagramı, Kilometre Taşları ve Proje Yönetimi Araçları
- 14. Örnek Sorular

1. Yazılım Mühendisliğinin Tanımına Dönüş

Hoca derse, bu haftanın planında bir değişiklik olduğunu söyleyerek başladı: Uygulama dersi bir hafta sonraya kaldı. Bu derste hem geçen haftanın konusu olan yazılım kalite yönetimi ve test tekrar edilecek, hem de yazılım proje yönetimi (software project management) konusuna geçilecekti.

Tekrara, ilk derslerde yapılan yazılım mühendisliği tanımıyla girildi. En kısa hâliyle yazılım mühendisliği, **kaliteli yazılımı en ekonomik şekilde elde etmek** için **mühendislik yaklaşımını, araçlarını ve yöntemlerini** kullanan bilim dalıdır. Bu tanımda üç anahtar kavram birleşir:

- **Kaliteli yazılım:** Herhangi bir yazılım değil, kaliteli bir yazılım hazırlanacak. Kaliteli yazılımdan beklentilerimiz geçen hafta anlatılmıştı.
- **En ekonomik şekilde elde etme:** Bu dersin yeni konusu olan proje yönetimiyle ilgilidir. Bir yanı maliyetin ekonomik tutulması ve maliyet-fayda analizinin yapılması, öbür yanı işin istenen zamanda ve istenen kapsamda bitmesidir.
- **Mühendislik araç ve yöntemlerinin kullanılması:** Hocaya göre mühendisliğin iki temel kavramı vardır: **tasarım** ve **ölçüm**.

Tasarım olmadan yazılımın kodunu oluşturmak diye bir şey yoktur. Tasarımla ilgili ayrıntılar bu derste nesneye yönelik tasarım kapsamında görüldü; tasarım kalıpları ve tekrar kullanılabilirlik (reusability) kavramı da işlendi. İkinci kavram ölçmedir. Hoca burada ilk derslerde söylediği iki cümleyi hatırlattı: “Ölçemediğinizi yönetemezsiniz” ve “Mühendis, ölçüm gerçekleştiren kişidir.”

Hocanın vurgusu: Mühendisliğin iki temel kavramı tasarım ve ölçümdür. Ölçemediğinizi yönetemezsiniz.

2. Yazılım Kalite Faktörleri ve McCall Modeli

2.1 Kalite faktörleri

Hoca, yazılım kalitesini oluşturan faktörleri ISO 9126'da görebileceğimizi söyleyerek bunları tek tek saydı:

- **Doğruluk (correctness):** Yazılımın spesifikasyonlara, yani ilk baştaki müşteri isteklerine uygun olması ve bu istekleri hangi derecede karşıladığı.
- **Güvenirlik (reliability):** İşlevlerin ne kadar hassasiyetle yerine getirildiği. Hoca bunun güvenlikle karıştırılmaması gerektiğini özellikle söyledi.
- **Verimlilik (efficiency), etkinlik:** Programın işlevlerini yerine getirmek için kullandığı kaynakların miktarı.
- **Güvenlik (security, integrity):** Yetkisiz kişilerin yazılıma müdahale etmesinin önlenmesi.
- **Kullanışlılık (usability, user friendly):** Öğrenmenin, işletmenin, girdi hazırlamanın ve çıktıyı yorumlamanın kolay olması.
- **Hata bulma kolaylığı:** Hatanın yerinin kolay bulunması ve düzeltilebilmesi.
- **Esneklik (flexibility):** Yazılımda kolay değişiklik yapabilme imkânı.
- **Sınama kolaylığı (testability):** Yazılımın takip edilmesinin ve test edilmesinin kolay olması.
- **Taşınabilirlik (portability):** Yazılımın bir sistemden başka bir sisteme kolay aktarılması.
- **Tekrar kullanılabilirlik (reusability):** Yazılımın tamamının ya da bir bölümünün farklı uygulamalarda kullanılabilmesi. Tasarım yapılırken zaten dikkate alınan bir özelliktir.
- **Birlikte çalışabilirlik (interoperability):** Başka bir sistemle kolay veri alışverişi yapabilme. Hoca, geçen hafta da söylediği gibi, bunun Türkçede "bağlanabilirlik" olarak da karşılandığını belirtti.

2.2 McCall'un üç bakış açısı

Bu faktörler McCall'un modelinde bir üçgenin üç kenarına yerleştirilir. Her kenar bir bakış açısıdır ve şu soruyu sorar: Bu bakış açısından kaliteli yazılım ne demektir?

- **Kullanıcı bakış açısı:** Doğruluk, güvenirlik, verimlilik, güvenlik ve kullanışlılık. Hoca bunu kendi deneyimimize bağladı: Bilgisayarınıza bir yazılım yüklediğinizde doğru çalışmasını, bu doğruluğun istenen duyarlılıkta olmasını, kaynakları iyi kullanmasını, başkalarının müdahale edemeyeceği şekilde güvenli olmasını ve öğrenmenin, girdi girmenin, çıktı yorumlamanın kolay olmasını istersiniz.
- **Yazılımı çalıştıranların (operatörlerin) bakış açısı:** Birlikte çalışabilirlik, taşınabilirlik ve tekrar kullanılabilirlik. Hoca, bazı kaynakların iç ve dış kalite özelliklerinden söz ettiğini, bu gruptakileri de iç kalite özelliği olarak saydığını ekledi.
- **Bakım yapanların bakış açısı:** Sınama kolaylığı, esneklik ve hata bulma kolaylığı. Bir hata çıktığında ya da başka bir nedenle bakım istendiğinde, kendi yazılımı ya da başkasının yazılımı üzerinde düzeltme yapacak yazılım uzmanlarının istediği özelliklerdir.

  Ders sonrası not: Altyazıdaki "MacCall/Maccol" doğru yazımıyla McCall'dur. Modelde üç grubun adları ürün işletimi (product operation), ürün geçişi (product transition) ve ürün revizyonudur (product revision). Hocanın "operatörlerin bakış açısı" dediği grup, yazılımın başka ortamlara aktarılmasıyla ilgili ürün geçişine karşılık gelir. Ayrıca hocanın ISO 9126 diye saydığı liste daha çok McCall faktörleridir; ISO/IEC 9126 büyük olasılıkla altı ana özellik tanımlar (işlevsellik, güvenilirlik, kullanılabilirlik, verimlilik, bakım yapılabilirlik, taşınabilirlik) ve yerini büyük olasılıkla 2011'de ISO/IEC 25010'a bırakmıştır.

2.3 Kalite faktörünün ölçülebilir hâle getirilmesi

Bu faktörler (hoca bunları F ile gösterdi) doğrudan ölçülemez; daha çok tanımsal değerlerdir. McCall'un yaklaşımında her kalite faktörü, onu etkileyen metriklerin belirli katsayılarla

ağırlıklandırılmış toplamı olarak ifade edilir:

$$F_q = c_1 m_1 + c_2 m_2 + \cdots + c_n m_n$$

Burada m_i faktörü ölçmeye yarayan metrikler, c_i ise bunların katsayılarıdır. Hoca bunu, kalite özelliğinin bir diskriminant fonksiyonuyla tahmin edilmesi olarak anlattı. Bir kalite faktörünün farklı metrikler aracılığıyla ölçülebilir hâle getirilmesine McCall'un kalite modeli denir.

3. Kaliteyi Sağlamanın Yolları

Kaliteli yazılımın ne olduğunu tanımladıktan sonra hoca şu soruyu sordu: Bir yazılımı geliştirirken kaliteli olmasını nasıl sağlayacağız? Önce tasarım iyi yapılmalı, gereksinim analizi sırasında müşteriyle iyi görüşülmeli ve bilgi alınmalıdır. Yani geliştirme süreci kurallara bağlı olmalıdır.

Geçen haftaki bir şekil üzerinden de hatırlatıldığı gibi, ilk aşamalarda yakalanan hatalar daha kolay ve daha az maliyetle düzeltilir. Bu yüzden kaliteyi sağlama yolları sırasıyla şunlardır:

- **1. Bir süreç modeli seçip kurallı uygulamak:** Waterfall, Scrum ya da başka bir model seçilir ve takip edilir. Analiz, tasarım ve kodlama aşamaları modelin kurallarına göre geçilir. Test aşamasında da performans ve doğruluk gibi faktörlerin ölçülmesine çaba gösterilir.
- **2. Kilometre taşlarında gözden geçirme (review):** Hatayı erken yakalamak için erken aşamalardan itibaren gözden geçirme yapılır. Kilometre taşları (milestones), süreçteki durak noktalarıdır; örneğin gereksinim analizinin ya da tasarımın bittiği anlar.
- **3. Aralarda inceleme (inspection):** Kilometre taşlarındaki gözden geçirmelerin arasında da inceleme yapılır. İnceleme, gözden geçirmenin daha kurallı ve daha yoğun yapılmasıdır.
- **4. Test (sınama):** Gözden geçirmelere rağmen programda hata kalmışsa, onu bulmak için test yapılır.

Üretim aşamasındaki bu kontrole kalite kontrolü denir. Kalite kontrolü önceden planlanmalıdır: Ne zaman ve hangi koşullarda yapılacağı, gözden geçirmeye kimlerin katılacağı belirlenmelidir. Aynı kontrol, kaynak program yazılırken de uygulanır.

> **Hocanın vurgusu:** Test yalnızca hatayı bulma aşamasıdır: hatanın yerini ve tipini bulur. Hatanın düzeltilmesi teste dahil değildir; hata ayıklama (debug) ayrıca yapılır.

Testte işlevsel hatalar (işlem hataları) bulunur, performans testleriyle performans hataları bulunur. Dayanıklılık testleri ve yapısal testlerle de yazılımın mantığı, yani mantıksal yollar sınanır.

4. Test Stratejisi ve Test Tipleri

4.1 Test basamakları

Yazılım test stratejisinde soru şudur: Geliştirilen bir yazılımı hangi sırayla test ederiz? Sıra şöyledir:

- **Birim testi (unit test):** Her kod birimi tamamlandığında, en küçük birim üzerinde yapılır. Daha çok kontrol yapıları, yani iç yapı test edilir.
- **Bütünleştirme (bütünlük) testi (integration test):** Test edilmiş birimler bir araya getirilir ve birleştiklerinde sorun çıkıp çıkmadığına bakılır. Bütünleştirme testi artırmalı (incremental) yapılır.
- **Sistem testi:** Sistem gereksinimlerine göre bakılır. Güvenlik, dayanıklılık ve performans testleri burada yapılır. Yazılımın veritabanı yönetim sistemi ve işletim sistemiyle ilişkileri de denenir.
- **Onaylama (kabul) testi (acceptance test):** Müşteri istekleri denenir; kullanıcı testidir.

Hoca, onaylama testi ile sistem testinin yer değiştirebileceğini söyledi; çünkü sistem testi yapılıp yazılım ondan sonra müşteriye teslim edilebilir.

> **Hocanın vurgusu:** Birim testini kodu yazan kişi yapar; test ekibi değil, geliştirme ekibi yapar. Bütünleştirme testini ise bazen test ekibi, bazen geliştirme ekibi yapar.

4.2 Test helezonu

Bu basamaklar bir helezon üzerinde gösterilir. En içte kodlama vardır ve kodlanan her birime birim testi yapılır. Kodlamadan önce tasarım yapılmıştır; tasarım bütünleştirme testiyle sınanır, yani modüllerin birbirine nasıl bağlandığına göre birleştirilip test edilir. Bunun dışında gereksinimler ve onaylama testi, en dışta ise sistem ve sistem testi yer alır.

4.3 Saydam kutu ve kara kutu testi

Hoca test tiplerini kara kutu, saydam kutu ve gri kutu testi olarak saydı.

Saydam kutu (white box) testinde iç yapı test edilir. İç yapının elemanları döngü yapıları, kontrol yapıları ve akış grafındaki yollardır. Akış grafında kaç yol varsa her birinin test edilmesi gerektiği için temel yol testi (basis path testing) ve döngü testi saydam kutu test tiplerindendir.

Kara kutu (black box) testinde dış yapı, iç yapı görülmeden test edilir. Sisteme girdiler verilir ve bu girdilere karşı istenen çıktıların elde edilip edilmediğine bakılır; modüllerin iç yapısı görünmez. Performans hatası, başlatma-bitirme hatası gibi hatalar da aranır. Asıl soru, girdilerin nasıl planlanacağıdır:

- **Eşdeğer bölümleme (equivalence partitioning):** Veri uzayı bölümlere ayrılır ve her bölümden bir veriyle test yapılır.
- **Sınır değer analizi (boundary value analysis):** "Hatalar genellikle sınırlardadır" düşüncesine dayanır; sınır değerleriyle test yapılır.

Bunun için graflar da oluşturulabilir; neden-sonuç grafı (cause-effect graph) bir seçenektir. Ama hocaya göre testte daha çok kullanılan kara kutu teknikleri eşdeğer bölümleme ve sınır değer analizidir.

5. Yazılım Mühendisliğinin Bölümleri ve Proje Yönetiminin Önemi

Tekrarın ardından hoca yeni bir ekran paylaşarak proje yönetimine girdi. Önce dersin genel yapısını hatırlattı. Yazılım mühendisliğinin temel kitapları olan Roger Pressman ve Ian Sommerville'in kitapları konuyu kısımlara ayırır:

- **Giriş:** Yazılım nedir, yazılım mühendisliği nedir, yazılımla ilgili sınıflamalar nelerdir; CMMI gibi yazılımın yetkinlik derecesini ölçen modeller; yazılım süreç modelleri.
- **Analiz, tasarım ve gerçekleştirme:** Hangi yöntemi izlediğimiz, gereksinim analizinde ve tasarımda hangi diyagramları neden ve neyi tarif etmek için kullandığımız. Bu derste bu kısım nesneye yönelik analiz ve tasarım kapsamında görüldü.
- **Kalite yönetimi ve test teknikleri:** Geçen hafta görüldü, bu hafta tekrar edildi.
- **Proje yönetimi:** Bu dersin konusu.

> **Ders sonrası not:** Altyazıdaki "Ayn Summerville" doğru yazımıyla Ian Sommerville, "SMMI" ise CMMI'dır (Capability Maturity Model Integration).

Hocaya göre özellikle 2000'li yıllardan itibaren başarısız yazılım projelerinin çok olduğu görüldü ve yazılım geliştirme ekiplerine proje yönetimi konusunda destek verilmeye başlandı. Bu alanda PMI (Project Management Institute) adlı bir enstitü vardır. Türkiye'deki PMI toplantılarına ve dergilerine bakıldığında katılımcıların en çok yazılım geliştiriciler olduğu görülür. Proje yönetimine ağırlık veren bir diğer grup inşaat mühendisleridir, çünkü onların da büyük projeleri vardır. Yazılım projeleri de büyüktür ve geçmişte başarısız olmuştur; bu yüzden başarı için proje yönetimine ağırlık verilmiştir.

Bu alanda PMP (Project Management Professional) sertifikaları vardır ve yazılım mühendisliği için özellikle önemlidir. Çevik (agile) yazılım geliştirme modelleri yaygınlaşınca çevik projeler için ayrı bir sertifika da verilmeye başlandı.

> **Ek bilgi (derste anlatılmadı):** Hocanın "agile PMP sertifikası" dediği sertifikanın adı büyük olasılıkla PMI-ACP'dir (PMI Agile Certified Practitioner). PMI'ın yayımladığı PMBOK rehberi, proje yönetimi bilgisinin temel başvuru kaynağıdır.

6. Proje Yönetiminin Aşamaları

Proje yönetimi, özellikle büyük projelerde şu soruların cevaplanmasıdır: Bu projeye hangi süreç modeli uygun? Projenin kapsamı nedir? Projenin katılımcıları, yani paydaşları kimlerdir? Zaman planı ve kaynak planı nedir? Bunlar belirlendikten sonra da her şey hep böyle kalacak değildir. Bir yerde başarısız olunursa plan dinamik olarak değişmelidir; çevik proje yönetiminde bu dinamikliğin sağlandığını görüyoruz.

> **Hocanın vurgusu:** Proje yönetimi, projeyi planlamaya eşit değildir. Proje planlama, proje yönetiminin yalnızca bir parçasıdır.

Proje yönetiminin aşamaları şunlardır:

- **Başlatma:** Projeye bir başlangıç yapmak.
- **Tanımlama:** Projeyi anlamak ve tanımlayabilmek.
- **Planlama:** Proje planını yapmak.
- **Yürütme ve takip:** Plan yapıldı diye iş bitmez; plan dinamik olabilir, bir B planı olabilir.
- **Kapanış:** Projenin sonuçlandırılması.

Takip aşamasını hoca bir örnekle anlattı. Bir proje önerisi veriliyor ve örneğin TÜBİTAK sponsor oluyor ya da Sanayi Bakanlığı projeyi desteklemeye layık görüyor. Proje iki yıl, bir buçuk yıl ya da altı ay sürebilir. Bu süre boyunca projeyi hem sponsor hem de içeriden proje ekibi takip eder. Takibin faydası şu sorularda görülür: Geldiğimiz yere kadar plan hedeflerimiz tuttu mu? Tutmadıysa planda neleri değiştireceğiz? Zaman planımız değişecek mi, öyle bir imkânımız var mı? Bu kontrol de proje yönetiminin bir parçasıdır.

Kapanış aşamasında proje çıktıları ölçülür, projenin arşiv bilgileri veritabanına aktarılır ve bu bilgiler daha sonra kullanılmak üzere ortak bir havuza atılır.

Hoca dersin sonuna doğru aynı yapıyı bir kez daha vurguladı: Proje yönetimi projeyi anlama, tanımlama, planlama, çalıştırma ve kapatma aşamalarından oluşur; “dört ya da beş adım” olarak sayılabilir ve planlama bunlardan yalnızca biridir.

> **Ek bilgi (derste anlatılmadı):** PMBOK rehberi proje yönetimini büyük olasılıkla beş süreç grubuna ayırır: başlatma (initiating), planlama (planning), yürütme (executing), izleme ve kontrol (monitoring and controlling), kapanış (closing). Hocanın “dört ya da beş adım” dediği yapı buna karşılık gelir.

6.1 Yazılım proje yönetiminin farkları

Yazılım proje yönetiminde önce projeye hangi geliştirme modelinin uygun olduğuna kafa yorulur, sonra hangi ölçümlerin yapılacağı belirlenir. Hoca, yazılım ekibinin diğer ekiplerden farkı olup olmadığını sordu ve şöyle cevapladı: Ekibin yönetiminde, organizasyonunda ve kaynak yönetiminde farklılıklar olabilir. Başka projelerde ekipman maliyeti ağır basarken yazılımda kafa işi, yani insan gücü öndedir. Bu yüzden personelle ilgili konular öne çıkar. Örneğin proje yönetiminin bir parçası olan risk yönetiminde de insan tarafı daha ağır basar. Genel proje yönetimini bilmek gerekir; genel tanımlar yazılım proje yönetiminde farklılık gösteriyorsa bunlar ayrıca işaret edilecektir.

7. Planlama İlkeleri ve Ölçüm

7.1 Planlama ilkeleri

Hoca proje planlamanın ilkelerini şöyle sıraladı:

- **Önce sınırları, yani kapsamı belirleyin.** Projede ne isteniyor ve sınırlar nerede? Gereksiz ayrıntılarla uğraşmayın; müşterinin istediğinin özünü kapsayın ve müşteriyle önce kapsam konusunda anlaşın.
- **Müşteriyi planlama eylemlerine katın.** Müşteri, planlama sırasında yapılacaklara adım adım onay versin.
- **Bildiklerinizi kullanarak tahmin yapın.** Tahminlerin doğru olması önemlidir.
- **Risk değerlendirmesi yapın ve gerçekçi olun.**
- **Ölçeği gözetin.** Nerede ince ayrıntıya gireceğinize karar verin. Büyük bir projede önce genel yapıyı kurup sonra her maddenin ayrıntısını düşünebilirsiniz.

- **Planın gidişini takip edin.** Her aşamada, yani kilometre taşlarında değerlendirme ve gözden geçirme yapın. Planınızın içinde bir kalite planı da olsun.
- **Değişikliği planlayın.** Müşteri değişiklik isteyecektir.

Son madde yazılım projelerinde özellikle önemlidir. Hocaya göre müşteri istediği yazılımı baştan kafasında canlandıramaz; sonradan eklemeler ve değişiklikler isteyebilir. Bu yüzden proje planında müşterinin hangi koşullarla değişiklik isteyebileceği ve bu değişiklikte yeni maliyetin nasıl gösterileceği kayıt altına alınmalıdır.

7.2 Ölçüm

Proje yönetiminde hem projenin hem de sürecin ölçülmesi sağlanabilir. Hoca ölçümü iki açıdan ele aldı:

- **Yazılımın (ürünün, projenin) ölçülmesi:** Yazılımın kalite özelliklerinin, özellikle iç kalite özelliklerinin ne durumda olduğuna bakılır. Proje ile ilgili ölçütler yazılımın iç yapısına ilişkin özellikleri ölçer.
- **Sürecin ölçülmesi:** Yazılım geliştirme sürecinin ölçülmesidir. Süreç ölçütleri şunlardır:
  - Hata sayısı: Belli bir süre kullanıldıktan sonra ne kadar hata çıktığı.
  - Hata giderme etkinliği: Hataları gidermek için neler yapıldığı.
  - Doğruluk: Hata sayısının proje boyutuna oranı.
  - Bakım kolaylığı: Bir hatanın bulunması ve giderilmesi için geçen süre.

Hangi ölçümlerin yapılacağına, kaliteyi etkileyen etkenlere bakılarak karar verilir: kişiler, ürün, teknoloji, çevre (iş) koşulları ve müşterinin özellikleri. Örneğin kişiler için hangi alanda çalışmak istedikleri ve hangi alanda daha başarılı oldukları ölçülmelidir.

Hoca bu kısmın ardından 10 dakika ara verdi.

8. Yazılım Proje Planının İçeriği

Aradan sonra hoca bir yazılım proje planında neler olması gerektiğini anlattı:

- **Projenin amacı, kapsamı ve projede yapılacak işler.** Bu işler, yazılımın hangi süreç modeliyle geliştirildiğine bağlıdır. Modelde hangi aşamalar varsa bunlar iş aktiviteleri olarak belirtilir. İş listesi, yani iş kırılım yapısı (Work Breakdown Structure, WBS), süreç modeline göre belirlenir.
- **Zaman planı ve kaynak planı:** WBS maddeleri üzerinden hangi işin hangi zamanda ve hangi kaynaklarla yapılacağı. Bunun takibi MS Project ile ya da Gantt diyagramıyla yapılabilir. Hoca, bunların geçen yıl sistem analizi dersinde zaman fizibilitesi kapsamında görüldüğünü hatırlattı.
- **Kalite planı ve risk planı.**
- **Kabuller (varsayımlar) ve kısıtlar.**
- **İş listesindeki öncelikler, zaman planı ve kilometre taşları.**

Hoca, bir yazılım proje planında hangi başlıkların olması gerektiğini tarif eden bir IEEE standardı bulunduğunu ve buna uyulması gerektiğini söyledi. Standart, planın içeriğini bir başlıklar listesi olarak verir.

> **Ek bilgi (derste anlatılmadı):** Hocanın andığı standart büyük olasılıkla IEEE 1058'dir (Standard for Software Project Management Plans). Bu standardın içeriği bugün büyük olasılıkla ISO/IEC/IEEE 16326 içinde yer almaktadır.

9. Risk Yönetimi

9.1 Risk nedir?

Hoca, proje planının içinde ve proje yönetiminin altında yer alması gereken risk yönetimine geçti. Risk tanımında üç temel öğe vardır:

- **Kayıp:** Risk gerçekleştiğinde istenmeyen sonuçlar doğar.
- **Gelecek:** Kayıp şu anda değil, gelecekte olacaktır. Şu anda var olan bir durum değildir.
- **Olasılık:** Riskin gerçekleşme yüzdesi vardır. Bir risk için "yüzde yüz gerçekleşecek" denemez; ama olasılığı yüksek ya da düşük olabilir.

Gerçekleştiğinde yıkımı daha fazla ya da daha az olabilir. Riskin önem derecesi, olasılığı ve bu yıkımın büyüklüğüne göre belirlenir.

Risklerle başa çıkmanın iki yolu vardır:

- Risk gerçekleşmeden önce önlem almak.
- Risk gerçekleşince uygulanacak çözüm planları, yani bir B planı hazırlamak.

Risk yönetim planları bunlar üzerine kurulur.

9.2 Risk türleri

- **Proje riskleri:** Projede yapılan işlerle ilgili risklerdir; bütçe riski, zaman riski, personel riski gibi. Örneğin önemli bir personelin işten ayrılması ya da bir personelin bazı temel bilgileri bilmemesinden doğan yetersizlik riski.
- **Teknik riskler:** Üretilen yazılımla ve yazılım geliştirme süreç modeliyle ilgili risklerdir. Örneğin yeni bir CASE aracı ya da yeni bir programlama dili kullanmanın riski.
- **İş (işletme) riskleri:** Pazar riski ve satış riski. Bir yazılım hazırlandı ama talep ne olacak gibi sorulardır.

Hocaya göre iş riskleri son yıllarda özellikle gündeme geldi. Bu yüzden yazılım firmalarında, diğer firmalarda olduğu gibi, hem pazarlama hem de yazılım bilgisine sahip kişilerin bulunması gerekir. Böylece hangi yazılımlara ihtiyaç olduğu ya da geliştirilen yazılımın hangi firmalara satılabileceği daha planlı ve sistemli düşünülebilir.

9.3 Risk tablosu

Risk tablosu şu adımlarla kurulur:

- Riskler tanımlanır.
- Her riskin kategorisi belirlenir. Örneğin bir personelin işten ayrılma olasılığı, personelle ilgili bir proje riskidir.
- Riskin olma olasılığı tahmin edilir: düşük, orta, yüksek.
- Riskin gerçekleşirse etkisi tahmin edilir: düşük (küçük), orta, yüksek (büyük).
- İkisinin birleşimi riskin büyüklüğünü, yani önemini verir. Hoca bunu “ikisinin çarpımı büyüklüğünü verecektir” diye anlattı.

Böylece başta risklerin derecelendirilmesi sağlanır: Hangi riskler sıkı takip edilmeli, hangileri yalnızca bilinip arada “bu risk hâlâ var mı” diye kontrol edilmeli?

9.4 Örnek özet risk tablosu

Hoca slaytta örnek bir özet risk tablosu gösterdi. Tabloda risk numarası, adı, türü, etkisi ve olasılığı vardı; altında da etki (küçük-büyük) ve olasılık (düşük-yüksek) eksenli bir diyagram yer alıyordu. Altyazıda yalnızca birkaç satır sözlü okunuyor: 1 numaralı risk organizasyonla ilgili, proje kategorisinde, etkisi orta, olasılığı yüksek; 2 numaralı risk projedeki tahminlerle ilgili tahmin riski, etkisi büyük, olasılığı düşük. Pazarlama riski için de olasılığın yüksek, etkinin büyük olduğu söyleniyor. Tablonun tamamı, aynı dersin kısa notunda slayt karesinden okunduğu biçimiyle şöyledir:

No	Ad	Tür	Etki	Olasılık
01	Organizasyon	Proje	Orta	Yüksek
02	Tahmin	Proje	Büyük	Düşük
03	Kullanışlılık	Teknik	Büyük	Düşük
04	Doğruluk	Teknik	Orta	Düşük
05	Güvenirlik	Teknik	Büyük	Düşük
06	Personel	Proje	Küçük	Düşük
07	Araçlar	Teknik	Orta	Orta
08	Metot	Teknik	Orta	Orta
09	Pazarlama	İş	Büyük	Yüksek

Hocanın diyagram üzerinden yaptığı yorum şöyledir:

- Etkisi küçük ve olasılığı düşük olan riskleri takip etmeye gerek yoktur. Örnekte bunlar 4, 6, 7 ve 8 numaralı risklerdir.
- Etkisi büyük ama olasılığı düşük olan ya da olasılığı yüksek ama etkisi küçük olan riskler takip edilir.
- Diğer bütün riskler için ayrıntılı bir risk bilgi sayfası hazırlanır.

  Ders sonrası not: Kısa nottaki tabloya göre 7 ve 8 numaralı risklerin etkisi ve olasılığı "Orta", 4 numaralının etkisi "Orta"dır. Hoca ise bu riskleri (6 ile birlikte) diyagramdaki "etkisi küçük, olasılığı düşük" bölgesinde sayıp izlenmeyeceğini söyledi. Tablodaki değerlerle bu sınıflandırma 4, 7 ve 8 için tutarlı değildir; özellikle 7 ve 8 (Orta/Orta), izlenmesine karar verilen 2, 3 ve 5'ten (Büyük/Düşük) daha düşük önemde görünmez. Slayttaki diyagram tablodan farklı çizilmiş olabilir.

9.5 Risklerin takibi ve risk bilgi sayfası

İzlenecek riskler kilometre taşlarında yeniden değerlendirilir. Bunun için özel bir tablo oluşturulur. Örneğin gereksinim analizi bittiğinde, yani o kilometre taşına gelindiğinde, şu sorular sorulur: 2 numaralı proje riskinin olasılığı hâlâ düşük mü, yükseldi mi, orta mı oldu? Etkisi aynı mı? Bunun için nasıl bir önlem gerekiyor? Yani her risk için daha ayrıntılı bir tablo, bir risk bilgi sayfası gerekir.

Hoca örnek olarak pazarlama riskinin bilgi sayfasını anlattı:

- **Risk:** Pazarlama riski; ticari (işletme) risklerinden. Bir pazarlama uzmanı yoktur. Olasılığı yüksek, etkisi büyüktür.
- **Durum:** Firmada yalnızca yazılım geliştiricilerden oluşan bir çekirdek kadro vardır, ama yazılımın piyasada çok fazla alternatifi vardır.
- **İşaretler:** Ürünün satışının yapılamaması; bakım aşamasında fiyatlandırmanın iyi yapılmaması. Bilgi sayfasında riskin her andaki durumu da izlenir.
- **Önlemler:**
  - Bir yazılımcıya pazarlama konusunda MBA yaptırmak.
  - Gereksinim mühendisliği aşamasında konu uzmanlığıyla ilgili danışmanlık hizmeti almak; yani yazılım hangi alanda geliştiriliyorsa o alanda bilgi sahibi olmak.
  - Pazarlamayı daha iyi yapabilmek için bir iki firmayı pilot müşteri olarak almak; bu firmalar alan bilgisi verdikleri için onlara daha uygun fiyata yazılım geliştirmek.

Önlemlerin hiçbiri bedava değildir; her birinin bir maliyeti vardır. Bu yüzden önlemler maliyetleriyle birlikte düşünülür: Hangi risk için hangi önlem alınacak, bu önlemin maliyeti ne, bu yola gidelim mi gitmeyelim mi?

Özetle risk yönetimi şöyle bir yol haritasıdır: Riskler tanımlanır, kategorileri bulunur, etki ve olasılıkları tahmin edilir. Etki ve olasılık birlikte düşünülerek riskin önemi belirlenir. Önem derecesine göre her riskin önlemine, sonra da her önlemin maliyetine bakılır.

Hoca, öğrencilerin projelerinde de projelerinin yapısına uygun bir risk tablosu oluşturmalarının isteneceğini söyledi.

9.6 Risk yönetimi ve olasılık-istatistik

Hoca sınıfa risk yönetiminin hangi bilim dalıyla yakından ilişkili olduğunu sordu. İlk öğrenci bilmediğini söyledi. İkinci öğrenci maliyet analizini önerdi. Hocanın cevabı **olasılık ve istatistik** oldu.

Gerekçesi şuydu: Risk yönetiminde en önemli şey, olma ihtimali olan bir olayın hangi kayıpları getireceğidir. Her riskin olasılığı ve etkisi tanımlanır; bunu belirleyebilmek için istatistikle ilgili değerlendirmeler gerekir. Bu yüzden yazılımda olduğu gibi genel olarak da bir risk değerlendirme uzmanının olasılık ve istatistik bilmesi gerekir. Hoca, veri biliminde ve veri madenciliğinde de istatistiğin çok kullanıldığını, bu nedenle bilgisayar mühendisliğinde temel istatistik ve istatistiksel veri analizi gibi derslerin mutlaka okutulduğunu ekledi.

10. Yazılım Ölçütleri (Software Metrics)

Yazılım üzerinde yapılması gereken mühendislik işlemlerinden biri de ölçümdür; bu, yazılım ölçütleri (software metrics) kavramıdır. Burada iki şey ayrılır: ölçme işlemi ve ölçmenin ne cinsinden yapıldığı. Hoca günlük hayattan örnekler verdi: Boy santimetreyle, hava sıcaklığı santigrat dereceyle ölçülür. Bir de derecelendirme vardır; örneğin ikinci sınavın ilk sınava göre ne kadar zor olduğu gibi.

Ölçüt (metrik), ölçmenin birimini gösterir: mesafe ölçütü, sıcaklık ölçütü gibi. Yazılımda da şu ölçütler vardır:

- **Karmaşıklık ölçütü:** Örneğin algoritma karmaşıklığı.
- **Büyüklük ölçütü:** Örneğin kod satırı sayısı.

Ölçme iki amaçla yapılır:

- Şu anda yapılan işin ve çalışan personelin kalitesini değerlendirmek.
- Bundan sonraki uygulamaların ve projelerin maliyet ve büyüklük tahminlerini daha doğru yapabilmek.

Hoca bu konuyu biraz daha açacağını söyleyerek ikinci arayı verdi. Dersin sonunda ise bu dönem büyüklük ölçümü ve büyüklük ölçüm sistemleri konusunda bilgi vermediğini, bunları projelerin sunumu sırasında arada anlatabileceğini söyledi. Yani bir projenin büyüklüğünün nasıl ölçüldüğü bu derste işlenmedi.

11. İş Planı ve İş Kırılım Yapısı

Arayı vermeden önce bir öğrenci projeyle ilgili bir soru sordu.

Soru: Projede iş planlarının sistem üzerinden yapılması isteniyor. İş planı olarak ne yapmamız gerekiyor?

Cevap: Hoca önce hangi süreç modelinin kullanıldığını sordu. Proje planı, başlangıçtan bitişe kadar hangi aktivitelerin olduğunu gösterir. Ekip çalışmasında iş ve görev paylaşımı yapılır, ama asıl soru iş adımlarının neler olduğudur: Proje waterfall ile mi, Scrum ile mi, yoksa hızlı uygulama geliştirme (Rapid Application Development) modeliyle mi ilerliyor? Her biri için iş aktiviteleri farklıdır; aynı işler yapılır ama farklı sırada ve farklı işleyişle yürür.

Waterfall kullanılıyorsa aktivite listesi en üst düzeyde analiz, tasarım ve gerçekleştirmedir. Bunlar hiyerarşik olarak WBS yapısıyla alt işlere bölünür:

```
1.0 Analiz (gereksinim analizi)
    1.1 Veri toplama
    1.2 Modelleme
    1.3 Yapılan işin raporunu yazma
2.0 Tasarım
    2.1 Genel tasarım (ön tasarım)
    2.2 Ayrıntılı tasarım
        2.2.1 Girdi tasarımı
        2.2.2 Çıktı tasarımı
        2.2.3 Veritabanı tasarımı
        2.2.4 İşlem tasarımı / arabirim tasarımı
3.0 Gerçekleştirme
```

Hoca, bu derste gereksinim analizinin 8 alt bölüme ayrıldığını, ama burada sistem analizi dersinde görüldüğü gibi kabaca 3 alt birime ayırdığını belirtti. İş kırılım listesi, gerçekleştirilecek işin hiyerarşik olarak alt maddelere bölünerek planlanmasıdır.

Scrum kullanılıyorsa işin ilerleyişi waterfall'dan farklıdır. Önce şu karar verilir: İşler ardışık mı, yani biri bitince öbürü mü başlıyor; yoksa iş önce parçalara ayrılıp her parça kendi içinde waterfall'a benzer biçimde mi ilerliyor? İş listesi bu karara göre oluşturulur.

12. Ayrıntılı Proje Planı Şablonu

Son aradan sonra hoca, öğrencilerin görmesini istediği ayrıntılı bir proje planı içerik şablonunu paylaştı ve başlıklar üzerinden geçti.

12.1 Giriş, amaç ve kapsam

Plan bir girişle ve proje planının amacıyla başlar. Sonra projenin genel amacı, projenin alanı (hangi alanda geliştirildiği), temel teslim edilecekler ve finansal beklentiler gelir. Hoca şablonun bu kısmının biraz fazla ayrıntılı göründüğünü, kısa kısa yazılabileceğini söyledi. Tamamlanma kriteri ise her zaman olmayabilir, ihmal edilebilir.

12.2 Kabuller ve kısıtlar

Kabuller, projenin geliştirilirken doğru varsayıldığı şeylerdir: "Bu proje şu kabullerle yapıldı." **Kısıtlar** ise projedeki sınırlamalardır; proje kısıtı ya da teknik kısıt olabilir. Hoca bunları risklere benzetti: Proje geliştirme süreçleriyle ilgili olabilirler ya da kullanılan teknik araçlarla. Örneğin personel kısıtı bir proje kısıtıdır. Bunlar kayda geçirilir.

12.3 İş planı

İş planı; teslim edilecekler, genel yaklaşım, WBS ve zaman listesinden oluşur. WBS, kullanılacak süreç modeline göre hiyerarşik yapıda kurulur; örneğin waterfall için 1.0 analiz, 2.0 tasarım ve bunların alt başlıkları. WBS'de dönüm noktası olan adımlar işaretlenir; bunlara kilometre taşı (milestone) denir. Ayrıca tahmin ve bütçeleme yapılır: zaman tahmini, kaynak tahmini ve nakit akışı analizi, yani ne kadar kaynak gerekeceği ve bu işten ne kadar kâr sağlanacağı.

12.4 Proje organizasyonu ve görevlendirme

Proje planında ekip yapısının nasıl kurulacağı ve görevlendirmenin nasıl yapılacağı da yer alır. Ekip yapısı üç türlü olabilir:

- **Şeflik sistemi:** Az kişi varsa uygundur.
- **Demokratik yapı.**
- **Hiyerarşik yapı:** Hocaya göre örneğin 10 kişiden sonrası demokratik ya da şeflik sistemiyle yürümez; hiyerarşik düzen daha doğrudur.

  Ek bilgi (derste anlatılmadı): Hocanın "şeflik sistemi" dediği yapı literatürde büyük olasılıkla baş programcı ekibi (chief programmer team), "demokratik yapı" ise egosuz programlama ekibi (egoless team) olarak geçer.

Ayrıntılı adımlar, yani ana adımlar ve WBS'deki alt başlıkların tanımları da bu kısımda verilir. Görevlendirme gereksinimleri de yazılır: sistem analisti, yazılım geliştirici, kullanıcı testi yapan gibi farklı iş gruplarından hangi yetenekler bekleniyor?

Projenin organizasyonu kısmında projedeki roller, sorumluluklar ve proje yöneticisi çevresinde nasıl bir yapılanma olduğu gösterilir.

12.5 Alan yönetimi ve iş listesi yönetimi

Alan (kapsam) yönetim planı, proje alanını, teslim edilecekleri ve projenin hedefini ayrıntılı olarak açıklar. Şablonda çok fazla alt başlık vardı; hoca asıl önemli olanın alandaki değişikliklerin nasıl yönetileceği olduğunu söyledi. Çünkü müşteri plan yapılırken bir şeyler istemiştir, ama gereksinim analizinin ayrıntılarını gördükçe daha fazlasını isteyebilir, yanlış anlattıklarını düzeltebilir ya da ek isteyebilir. Bu değişikliklerin onayının nasıl verileceği ve alan değişikliklerinin nasıl tanımlanacağı belirtilmelidir.

İş listesi yönetim planı, iş listesinde kaymalar olduğunda bunların nasıl yönetileceğini anlatır. Örneğin bir zaman kayması yaşandığında hangi önlemlerin alınacağı planda yer alabilir.

12.6 Maliyet, risk, iletişim ve kalite yönetimi

- **Maliyet yönetim planı:** Hangi maliyet kalemlerinin olduğu, yani bütçenin oluşturulması.
- **Risk yönetim planı:** Az önce anlatılan risklerin planı.
- **İletişim yönetimi:** Ekip içindeki iletişimin nasıl kademelendirileceği. Hocaya göre burada özellikle kastedilen, kötü bir durum senaryosu ortaya çıktığında bundan önceden haberdar olunmasını sağlayacak bir iletişim planıdır.
- **Kalite yönetim planı:** Kaliteyi sağlamak için yapılacak gözden geçirmelerin zamanı ve bunlara kimlerin katılacağı. Gözden geçirmelerin yalnızca kilometre taşlarında mı, başka zamanlarda da mı yapılacağı belirtilir. Kalite yönetimindeki hedefler ve ölçüler de belirlenir.
- **Satın alma ve faturalandırma:** Ürünle ilgili alımlar, kaynakların elde edilme planları ve danışmanlarla, alt yüklenicilerle yapılan anlaşmalardaki ücret ödemelerinin faturalandırma kuralları.

12.7 Bir proje planında mutlaka olması gerekenler

Hoca, şablonun çok kapsamlı olduğunu söyleyerek mutlaka olması gerekenleri özetledi. Proje planının amacının ayrıca yazılmasına gerek yoktur; ama şunlar olmalıdır:

- Proje amacı, proje alanı ve teslim edilecekler.
- Kabuller ve kısıtlar.
- Proje iş listesi: WBS ve alt başlıklar.
- Gantt diyagramı. MS Project ya da başka bir araçla hazırlanabilir.
- Alan yönetimi: alandaki değişikliklerin nasıl yapılacağı.
- İş listesindeki kaymaların nasıl yönetileceği.
- Bütçenin nasıl oluşturulacağı.
- Risk yönetim planı.
- Kalite yönetim planı: kalite gözden geçirmelerinde kimlerin olacağı ve hangi tarihlerde yapılacağı.
- Proje organizasyonu: ekibin hiyerarşik yapısı.

  Hocanın vurgusu: Hoca özellikle bu ayrıntılı proje planı yapısını görmeleri için öğrencilerin dikkatini çekti; projelerde risk tablosu, WBS ve Gantt diyagramı istenecek.

13. Gantt Diyagramı, Kilometre Taşları ve Proje Yönetimi Araçları

Ders sonunda hoca, sistem analizi dersini almayanlar için kilometre taşının ve Gantt diyagramının ne olduğunu örnek bir Gantt diyagramı üzerinden hatırlattı.

Gantt diyagramında iş listesi, yani WBS yapısı satırlarda görülür. Temel iş adımlarının altında alt adımlar listelenir. Kendi altında başka kırılımlar olan işler koyu renkte gösterilir. Kilometre taşları eşkenar dörtgenle gösterilir. Hocanın gösterdiği örnekte bir temel iş adımının alt başlıklarından birinin de kendi alt başlıkları vardı; bu alt başlıklardan sonra bir kilometre taşı, ardından da bir sonraki iş parçası geliyordu.

Proje yönetimi yazılımları çok sayıdadır. MS Project bunlardan biridir, ama birçok açık kaynaklı yazılım da vardır. Bu yazılımlar, iş kırılım listesi ve bilgiler girildiğinde Gantt ve PERT diyagramlarının çizilmesine yardım eder. Ayrıca planla gerçekleşenin karşılaştırılmasını ve kaynakların takibini sağlar. Hoca, özellikle sistem analizi dersini almayanlara bu araçları gözden geçirmelerini tavsiye etti.

14. Örnek Sorular

Bu derste kare yoktur; sorular derste sözlü sorulan sorulardan alınmıştır.

Soru 1 (derste sözlü, 01:14:26): Risk yönetimi gerçekleştirirken hangi bilim dalına daha çok ihtiyacımız vardır?

> **Çözüm (derste verildi):** Olasılık ve istatistik. Risk yönetiminde her riskin gerçekleşme olasılığı ve gerçekleşirse etkisi tahmin edilir. Bu tahminler istatistiksel değer-

lendirme gerektirir. Bir öğrencinin önerdiği maliyet analizi, önlemlerin maliyetini değerlendirirken kullanılır, ama hocanın beklediği cevap olasılık ve istatistikti.

Soru 2 (derste sözlü, 01:20:23): Waterfall modeliyle geliştirilen bir projenin iş planı, iş kırılım yapısı (WBS) olarak nasıl kurulur?

> **Çözüm (derste verildi):** Önce süreç modeli belirlenir. Waterfall'da en üst düzey işler 1.0 Analiz, 2.0 Tasarım, 3.0 Gerçekleştirme'dir. Analiz; 1.1 Veri toplama, 1.2 Modelleme, 1.3 Rapor yazma olarak bölünür. Tasarım; 2.1 Genel (ön) tasarım ve 2.2 Ayrıntılı tasarım olarak bölünür. Ayrıntılı tasarımın altında 2.2.1 Girdi, 2.2.2 Çıktı, 2.2.3 Veritabanı ve 2.2.4 İşlem/arabirim tasarımı yer alır (tam liste 11. bölümde). Scrum'da ise iş önce parçalara ayrılır ve iş listesi buna göre kurulur.

Dersin Toparlaması

Ders iki parçadan oluştu. İlk parçada geçen haftanın kalite konusu tekrar edildi: yazılım mühendisliğinin "kaliteli yazılım, en ekonomik şekilde, mühendislik araç ve yöntemleriyle" tanımı, McCall'un üç bakış açısına göre kalite faktörleri, kaliteyi sağlamanın sırası (süreç modeli, gözden geçirme, inceleme, test) ve test basamakları ile test tipleri. İkinci parçada tanımın "en ekonomik" kısmına karşılık gelen yazılım proje yönetimi işlendi. Temel mesaj, proje yönetiminin planlamadan ibaret olmadığıydı: Başlatma, tanımlama, planlama, yürütme-takip ve kapanış aşamalarının hepsi proje yönetimidir ve plan dinamik olarak kilometre taşlarında yeniden değerlendirilir.

Proje planının içeriği (amaç, kapsam, WBS, Gantt diyagramı, kabuller ve kısıtlar, alan ve kayma yönetimi, bütçe, kalite ve risk planları, ekip organizasyonu) ve risk yönetimi (risk tanımı, türleri, risk tablosu, olasılık-etki değerlendirmesi, risk bilgi sayfası ve önlemlerin maliyeti) ayrıntılı anlatıldı. Hoca bu dersin konu anlatımının son dersi olduğunu söyledi; büyüklük ölçüm yöntemlerinin bu dönem işlenmediğini, bunları proje sunumları sırasında arada anlatabileceğini ekledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.